JavaScript is disabled on your browser.

compact1, compact2, compact3

java.time

Class LocalDateTime

java.lang.Object
    java.time.LocalDateTime

All Implemented Interfaces:

`Serializable`, `Comparable<ChronoLocalDateTime<?>>`, `ChronoLocalDateTime<LocalDate>`, `Temporal`, `TemporalAccessor`, `TemporalAdjuster`

```
public final class LocalDateTime
extends Object
implements Temporal, TemporalAdjuster, ChronoLocalDateTime<LocalDate>, Serializable
```

A date-time without a time-zone in the ISO-8601 calendar system, such as `2007-12-03T10:15:30`.

`LocalDateTime` is an immutable date-time object that represents a date-time, often viewed as year-month-day-hour-minute-second. Other date and time fields, such as day-of-year, day-of-week and week-of-year, can also be accessed. Time is represented to nanosecond precision. For example, the value "2nd October 2007 at 13:45.30.123456789" can be stored in a `LocalDateTime`.

This class does not store or represent a time-zone. Instead, it is a description of the date, as used for birthdays, combined with the local time as seen on a wall clock. It cannot represent an instant on the time-line without additional information such as an offset or time-zone.

The ISO-8601 calendar system is the modern civil calendar system used today in most of the world. It is equivalent to the proleptic Gregorian calendar system, in which today's rules for leap years are applied for all time. For most applications written today, the ISO-8601 rules are entirely suitable. However, any application that makes use of historical dates, and requires them to be accurate will find the ISO-8601 approach unsuitable.

This is a value-based class; use of identity-sensitive operations (including reference equality (==), identity hash code, or synchronization) on instances of `LocalDateTime` may have unpredictable results and should be avoided. The `equals` method should be used for comparisons.

Implementation Requirements:

`This class is immutable and thread-safe.`

Since:

`1.8`

See Also:

`Serialized Form`

Field Summary

Fields

Modifier and Type	Field	Description
`static LocalDateTime`	`MAX`	The maximum supported `LocalDateTime`, '+999999999-12-31T23:59:59.999999999'.
`static LocalDateTime`	`MIN`	The minimum supported `LocalDateTime`, '-999999999-01-01T00:00:00'.

Method Summary

Concrete Methods

Modifier and Type	Method	Description
`Temporal`	`adjustInto(Temporal temporal)`	Adjusts the specified temporal object to have the same date and time as this object
`OffsetDateTime`	`atOffset(ZoneOffset offset)`	Combines this date-time with an offset to create an `OffsetDateTime`.
`ZonedDateTime`	`atZone(ZoneId zone)`	Combines this date-time with a time-zone to create a `ZonedDateTime`.
`int`		

	compareTo(**ChronoLocalDateTime**<? > other)	Compares this date-time to another date-time.
boolean	**equals**(**Object** obj)	Checks if this date-time is equal to another date-time.
String	**format**(**DateTimeFormatter** formatter)	Formats this date-time using the specified formatter.
static **LocalDateTime**	**from**(**TemporalAccessor** temporal)	Obtains an instance of `LocalDateTime` from a temporal object.
int	**get**(**TemporalField** field)	Gets the value of the specified field from this date-time as an `int`.
int	**getDayOfMonth**()	Gets the day-of-month field.
DayOfWeek	**getDayOfWeek**()	Gets the day-of-week field, which is an enum `DayOfWeek`.
int	**getDayOfYear**()	Gets the day-of-year field.
int	**getHour**()	Gets the hour-of-day field.
long	**getLong**(**TemporalField** field)	Gets the value of the specified field from this date-time as a `long`.
int	**getMinute**()	Gets the minute-of-hour field.
Month	**getMonth**()	Gets the month-of-year field using the `Month` enum.
int	**getMonthValue**()	Gets the month-of-year field from 1 to 12.

int	**getNano**()	Gets the nano-of-second field.
int	**getSecond**()	Gets the second-of-minute field.
int	**getYear**()	Gets the year field.
int	**hashCode**()	A hash code for this date-time.
boolean	**isAfter**(**ChronoLocalDateTime**<?> other)	Checks if this date-time is after the specified date-time.
boolean	**isBefore**(**ChronoLocalDateTime**<?> other)	Checks if this date-time is before the specified date-time.
boolean	**isEqual**(**ChronoLocalDateTime**<?> other)	Checks if this date-time is equal to the specified date-time.
boolean	**isSupported**(**TemporalField** field)	Checks if the specified field is supported.
boolean	**isSupported**(**TemporalUnit** unit)	Checks if the specified unit is supported.
LocalDateTime	**minus**(long amountToSubtract, **TemporalUnit** unit)	Returns a copy of this date-time with the specified amount subtracted.
LocalDateTime	**minus**(**TemporalAmount** amountToSubtract)	Returns a copy of this date-time with the specified amount subtracted.
LocalDateTime	**minusDays**(long days)	Returns a copy of this `LocalDateTime` with the specified number of days subtracted.
LocalDateTime	**minusHours**(long hours)	Returns a copy of this `LocalDateTime` with the specified number of hours subtracted.
LocalDateTime	**minusMinutes**(long minutes)	

		Returns a copy of this LocalDateTime with the specified number of minutes subtracted.
LocalDateTime	minusMonths(long months)	Returns a copy of this LocalDateTime with the specified number of months subtracted.
LocalDateTime	minusNanos(long nanos)	Returns a copy of this LocalDateTime with the specified number of nanoseconds subtracted.
LocalDateTime	minusSeconds(long seconds)	Returns a copy of this LocalDateTime with the specified number of seconds subtracted.
LocalDateTime	minusWeeks(long weeks)	Returns a copy of this LocalDateTime with the specified number of weeks subtracted.
LocalDateTime	minusYears(long years)	Returns a copy of this LocalDateTime with the specified number of years subtracted.
static LocalDateTime	now()	Obtains the current date-time from the system clock in the default time-zone.
static LocalDateTime	now(Clock clock)	Obtains the current date-time from the specified clock.
static LocalDateTime	now(ZoneId zone)	Obtains the current date-time from the system clock in the specified time-zone.
static LocalDateTime	of(int year, int month, int dayOfMonth, int hour, int minute)	Obtains an instance of LocalDateTime from year, month, day, hour and minute, setting the

		second and nanosecond to zero.
static **LocalDateTime**	**of**(int year, int month, int dayOfMonth, int hour, int minute, int second)	Obtains an instance of LocalDateTime from year, month, day, hour, minute and second, setting the nanosecond to zero.
static **LocalDateTime**	**of**(int year, int month, int dayOfMonth, int hour, int minute, int second, int nanoOfSecond)	Obtains an instance of LocalDateTime from year, month, day, hour, minute, second and nanosecond.
static **LocalDateTime**	**of**(int year, **Month** month, int dayOfMonth, int hour, int minute)	Obtains an instance of LocalDateTime from year, month, day, hour and minute, setting the second and nanosecond to zero.
static **LocalDateTime**	**of**(int year, **Month** month, int dayOfMonth, int hour, int minute, int second)	Obtains an instance of LocalDateTime from year, month, day, hour, minute and second, setting the nanosecond to zero.
static **LocalDateTime**	**of**(int year, **Month** month, int dayOfMonth, int hour, int minute, int second, int nanoOfSecond)	Obtains an instance of LocalDateTime from year, month, day, hour, minute, second and nanosecond.
static **LocalDateTime**	**of**(**LocalDate** date, **LocalTime** time)	Obtains an instance of LocalDateTime from a date and time.
static **LocalDateTime**	**ofEpochSecond**(long epochSecond, int nanoOfSecond, **ZoneOffset** offset)	Obtains an instance of LocalDateTime using seconds from the epoch of 1970-01-01T00:00:00Z

static **LocalDateTime**	**ofInstant**(**Instant** instant, **ZoneId** zone)	Obtains an instance of LocalDateTime from an Instant and zone ID.
static **LocalDateTime**	**parse**(**CharSequence** text)	Obtains an instance of LocalDateTime from a text string such as 2007-12-03T10:15:30.
static **LocalDateTime**	**parse**(**CharSequence** text, **DateTimeFormatter** formatter)	Obtains an instance of LocalDateTime from a text string using a specific formatter.
LocalDateTime	**plus**(long amountToAdd, **TemporalUnit** unit)	Returns a copy of this date-time with the specified amount added.
LocalDateTime	**plus**(**TemporalAmount** amountToAdd)	Returns a copy of this date-time with the specified amount added.
LocalDateTime	**plusDays**(long days)	Returns a copy of this LocalDateTime with the specified number of days added.
LocalDateTime	**plusHours**(long hours)	Returns a copy of this LocalDateTime with the specified number of hours added.
LocalDateTime	**plusMinutes**(long minutes)	Returns a copy of this LocalDateTime with the specified number of minutes added.
LocalDateTime	**plusMonths**(long months)	Returns a copy of this LocalDateTime with the specified number of months added.
LocalDateTime	**plusNanos**(long nanos)	Returns a copy of this LocalDateTime with the specified number of nanoseconds added.

LocalDateTime	**plusSeconds**(long seconds)	Returns a copy of this LocalDateTime with the specified number of seconds added.
LocalDateTime	**plusWeeks**(long weeks)	Returns a copy of this LocalDateTime with the specified number of weeks added.
LocalDateTime	**plusYears**(long years)	Returns a copy of this LocalDateTime with the specified number of years added.
<R> R	**query**(**TemporalQuery**<R> query)	Queries this date-time using the specified query.
ValueRange	**range**(**TemporalField** field)	Gets the range of valid values for the specified field.
LocalDate	**toLocalDate**()	Gets the LocalDate part of this date-time.
LocalTime	**toLocalTime**()	Gets the LocalTime part of this date-time.
String	**toString**()	Outputs this date-time as a String, such as 2007-12-03T10:15:30
LocalDateTime	**truncatedTo**(**TemporalUnit** unit)	Returns a copy of this LocalDateTime with the time truncated.
long	**until**(**Temporal** endExclusive, **TemporalUnit** unit)	Calculates the amount of time until another date-time in terms of the specified unit.
LocalDateTime	**with**(**TemporalAdjuster** adjuster)	Returns an adjusted copy of this date-time.
LocalDateTime	**with**(**TemporalField** field, long newValue)	Returns a copy of this date-time with the specified field set to a new value.

LocalDateTime	withDayOfMonth(int dayOfMonth)	Returns a copy of this LocalDateTime with the day-of-month altered.
LocalDateTime	withDayOfYear(int dayOfYear)	Returns a copy of this LocalDateTime with the day-of-year altered
LocalDateTime	withHour(int hour)	Returns a copy of this LocalDateTime with the hour-of-day altered.
LocalDateTime	withMinute(int minute)	Returns a copy of this LocalDateTime with the minute-of-hour altered.
LocalDateTime	withMonth(int month)	Returns a copy of this LocalDateTime with the month-of-year altered.
LocalDateTime	withNano(int nanoOfSecond)	Returns a copy of this LocalDateTime with the nano-of-second altered.
LocalDateTime	withSecond(int second)	Returns a copy of this LocalDateTime with the second-of-minute altered.
LocalDateTime	withYear(int year)	Returns a copy of this LocalDateTime with the year altered.

Methods inherited from class java.lang.Object

clone, finalize, getClass, notify, notifyAll, wait, wait, wait

Methods inherited from interface java.time.chrono.ChronoLocalDateTime

getChronology, timeLineOrder, toEpochSecond, toInstant

Field Detail

MIN

```
public static final LocalDateTime MIN
```

The minimum supported `LocalDateTime`, '-999999999-01-01T00:00:00'. This is the local date-time of midnight at the start of the minimum date. This combines `LocalDate.MIN` and `LocalTime.MIN`. This could be used by an application as a "far past" date-time.

MAX

```
public static final LocalDateTime MAX
```

The maximum supported `LocalDateTime`, '+999999999-12-31T23:59:59.999999999'. This is the local date-time just before midnight at the end of the maximum date. This combines `LocalDate.MAX` and `LocalTime.MAX`. This could be used by an application as a "far future" date-time.

Method Detail

now

```
public static LocalDateTime now()
```

Obtains
the current date-time from the system clock in the default time-zone.

This will query the `system clock` in the default time-zone to obtain the current date-time.

Using this method will prevent the ability to use an alternate clock for testing because the clock is hard-coded.

Returns:

`the current date-time using the system clock and default time-zone, not null`

now

```
public static LocalDateTime now(ZoneId zone)
```

Obtains the current date-time from the system clock in the specified time-zone.

This will query the `system clock` to obtain the current date-time. Specifying the time-zone avoids dependence on the default time-zone.

Using this method will prevent the ability to use an alternate clock for testing because the clock is hard-coded.

Parameters:

`zone - the zone ID to use, not null`

Returns:

`the current date-time using the system clock, not null`

now

```
public static LocalDateTime now(Clock clock)
```

Obtains the current date-time from the specified clock.

This will query the specified clock to obtain the current date-time. Using this method allows the use of an alternate clock for testing. The alternate clock may be introduced using `dependency injection`.

Parameters:

`clock - the clock to use, not null`

Returns:

`the current date-time, not null`

of

```
public static LocalDateTime of(int year,
                               Month month,
                               int dayOfMonth,
                               int hour,
                               int minute)
```

Obtains an instance of `LocalDateTime` from year, month, day, hour and minute, setting the second and nanosecond to zero.

This returns a `LocalDateTime` with the specified year, month, day-of-month, hour and minute. The day must be valid for the year and month, otherwise an exception will be thrown. The second and nanosecond fields will be set to zero.

Parameters:

`year - the year to represent, from MIN_YEAR to MAX_YEAR`

month - the month-of-year to represent, not null

dayOfMonth - the day-of-month to represent, from 1 to 31

hour - the hour-of-day to represent, from 0 to 23

minute - the minute-of-hour to represent, from 0 to 59

Returns:

the local date-time, not null

Throws:

DateTimeException - if the value of any field is out of range, or if the day-of-month is invalid for the month-year

of

```
public static LocalDateTime of(int year,
                               Month month,
                               int dayOfMonth,
                               int hour,
                               int minute,
                               int second)
```

Obtains an instance of LocalDateTime from year, month, day, hour, minute and second, setting the nanosecond to zero.

This returns a LocalDateTime with the specified year, month, day-of-month, hour, minute and second. The day must be valid for the year and month, otherwise an exception will be thrown. The nanosecond field will be set to zero.

Parameters:

year - the year to represent, from MIN_YEAR to MAX_YEAR

month - the month-of-year to represent, not null

dayOfMonth - the day-of-month to represent, from 1 to 31

hour - the hour-of-day to represent, from 0 to 23

minute - the minute-of-hour to represent, from 0 to 59

second - the second-of-minute to represent, from 0 to 59

Returns:

the local date-time, not null

Throws:

DateTimeException - if the value of any field is out of range, or if the day-of-month is invalid for the month-year

of

```
public static LocalDateTime of(int year,
                               Month month,
                               int dayOfMonth,
                               int hour,
                               int minute,
                               int second,
                               int nanoOfSecond)
```

Obtains an instance of `LocalDateTime` from year, month, day, hour, minute, second and nanosecond.

This returns a `LocalDateTime` with the specified year, month, day-of-month, hour, minute, second and nanosecond. The day must be valid for the year and month, otherwise an exception will be thrown.

Parameters:

`year` - the year to represent, from MIN_YEAR to MAX_YEAR

`month` - the month-of-year to represent, not null

`dayOfMonth` - the day-of-month to represent, from 1 to 31

`hour` - the hour-of-day to represent, from 0 to 23

`minute` - the minute-of-hour to represent, from 0 to 59

`second` - the second-of-minute to represent, from 0 to 59

`nanoOfSecond` - the nano-of-second to represent, from 0 to 999,999,999

Returns:

the local date-time, not null

Throws:

`DateTimeException` - if the value of any field is out of range, or if the day-of-month is invalid for the month-year

of

```
public static LocalDateTime of(int year,
                               int month,
                               int dayOfMonth,
                               int hour,
                               int minute)
```

Obtains an instance of `LocalDateTime` from year, month, day, hour and minute, setting the second and nanosecond to zero.

This returns a `LocalDateTime` with the specified year, month, day-of-month, hour and minute. The day must be valid for the year and month, otherwise an exception will be thrown. The second and nanosecond fields will be set to zero.

Parameters:

`year - the year to represent, from MIN_YEAR to MAX_YEAR`

`month - the month-of-year to represent, from 1 (January) to 12 (December)`

`dayOfMonth - the day-of-month to represent, from 1 to 31`

`hour - the hour-of-day to represent, from 0 to 23`

`minute - the minute-of-hour to represent, from 0 to 59`

Returns:

`the local date-time, not null`

Throws:

`DateTimeException - if the value of any field is out of range, or if the day-of-month is invalid for the month-year`

of

```
public static LocalDateTime of(int year,
                               int month,
                               int dayOfMonth,
                               int hour,
                               int minute,
                               int second)
```

Obtains an instance of `LocalDateTime` from year, month, day, hour, minute and second, setting the nanosecond to zero.

This returns a `LocalDateTime` with the specified year, month, day-of-month, hour, minute and second. The day must be valid for the year and month, otherwise an exception will be thrown. The nanosecond field will be set to zero.

Parameters:

`year - the year to represent, from MIN_YEAR to MAX_YEAR`

`month - the month-of-year to represent, from 1 (January) to 12 (December)`

`dayOfMonth - the day-of-month to represent, from 1 to 31`

`hour - the hour-of-day to represent, from 0 to 23`

`minute - the minute-of-hour to represent, from 0 to 59`

`second - the second-of-minute to represent, from 0 to 59`

Returns:

the local date-time, not null

Throws:

DateTimeException - if the value of any field is out of range, or if the day-of-month is invalid for the month-year

of

```
public static LocalDateTime of(int year,
                               int month,
                               int dayOfMonth,
                               int hour,
                               int minute,
                               int second,
                               int nanoOfSecond)
```

Obtains an instance of LocalDateTime from year, month, day, hour, minute, second and nanosecond.

This returns a LocalDateTime with the specified year, month, day-of-month, hour, minute, second and nanosecond. The day must be valid for the year and month, otherwise an exception will be thrown.

Parameters:

year - the year to represent, from MIN_YEAR to MAX_YEAR

month - the month-of-year to represent, from 1 (January) to 12 (December)

dayOfMonth - the day-of-month to represent, from 1 to 31

hour - the hour-of-day to represent, from 0 to 23

minute - the minute-of-hour to represent, from 0 to 59

second - the second-of-minute to represent, from 0 to 59

nanoOfSecond - the nano-of-second to represent, from 0 to 999,999,999

Returns:

the local date-time, not null

Throws:

DateTimeException - if the value of any field is out of range, or if the day-of-month is invalid for the month-year

of

```
public static LocalDateTime of(LocalDate date,
                               LocalTime time)
```

Obtains an instance of `LocalDateTime` from a date and time.

Parameters:

`date - the local date, not null`

`time - the local time, not null`

Returns:

`the local date-time, not null`

ofInstant

```
public static LocalDateTime ofInstant(Instant instant,
                                      ZoneId zone)
```

Obtains an instance of `LocalDateTime` from an `Instant` and zone ID.

This creates a local date-time based on the specified instant. First, the offset from UTC/Greenwich is obtained using the zone ID and instant, which is simple as there is only one valid offset for each instant. Then, the instant and offset are used to calculate the local date-time.

Parameters:

`instant - the instant to create the date-time from, not null`

`zone - the time-zone, which may be an offset, not null`

Returns:

`the local date-time, not null`

Throws:

`DateTimeException - if the result exceeds the supported range`

ofEpochSecond

```
public static LocalDateTime ofEpochSecond(long epochSecond,
                                          int nanoOfSecond,
                                          ZoneOffset offset)
```

Obtains an instance of `LocalDateTime` using seconds from the epoch of 1970-01-01T00:00:00Z.

This allows the epoch-second field to be converted to a local date-time. This is primarily intended for low-level conversions rather than general application usage.

Parameters:

epochSecond - the number of seconds from the epoch of 1970-01-01T00:00:00Z

nanoOfSecond - the nanosecond within the second, from 0 to 999,999,999

offset - the zone offset, not null

Returns:

the local date-time, not null

Throws:

DateTimeException - if the result exceeds the supported range, or if the nano-of-second is invalid

from

```
public static LocalDateTime from(TemporalAccessor temporal)
```

Obtains an instance of LocalDateTime from a temporal object.

This obtains a local date-time based on the specified temporal. A TemporalAccessor represents an arbitrary set of date and time information, which this factory converts to an instance of LocalDateTime.

The conversion extracts and combines the LocalDate and the LocalTime from the temporal object. Implementations are permitted to perform optimizations such as accessing those fields that are equivalent to the relevant objects.

This method matches the signature of the functional interface TemporalQuery allowing it to be used as a query via method reference, LocalDateTime::from.

Specified by:

from in interface ChronoLocalDateTime<LocalDate>

Parameters:

temporal - the temporal object to convert, not null

Returns:

the local date-time, not null

Throws:

DateTimeException - if unable to convert to a LocalDateTime

See Also:

Chronology.localDateTime(TemporalAccessor)

parse

```
public static LocalDateTime parse(CharSequence text)
```

Obtains an instance of LocalDateTime from a text string such as 2007-12-03T10:15:30.

The string must represent a valid date-time and is parsed using DateTimeFormatter.ISO_LOCAL_DATE_TIME.

Parameters:

text - the text to parse such as "2007-12-03T10:15:30", not null

Returns:

the parsed local date-time, not null

Throws:

DateTimeParseException - if the text cannot be parsed

parse

```
public static LocalDateTime parse(CharSequence text,
                                  DateTimeFormatter formatter)
```

Obtains an instance of LocalDateTime from a text string using a specific formatter.

The text is parsed using the formatter, returning a date-time.

Parameters:

text - the text to parse, not null

formatter - the formatter to use, not null

Returns:

the parsed local date-time, not null

Throws:

DateTimeParseException - if the text cannot be parsed

isSupported

```
public boolean isSupported(TemporalField field)
```

Checks if the specified field is supported.

This checks if this date-time can be queried for the specified field. If false, then calling the `range`, `get` and `with(TemporalField, long)` methods will throw an exception.

If the field is a `ChronoField` then the query is implemented here. The supported fields are:

- `NANO_OF_SECOND`
- `NANO_OF_DAY`
- `MICRO_OF_SECOND`
- `MICRO_OF_DAY`
- `MILLI_OF_SECOND`
- `MILLI_OF_DAY`
- `SECOND_OF_MINUTE`
- `SECOND_OF_DAY`
- `MINUTE_OF_HOUR`
- `MINUTE_OF_DAY`
- `HOUR_OF_AMPM`
- `CLOCK_HOUR_OF_AMPM`
- `HOUR_OF_DAY`
- `CLOCK_HOUR_OF_DAY`
- `AMPM_OF_DAY`
- `DAY_OF_WEEK`
- `ALIGNED_DAY_OF_WEEK_IN_MONTH`
- `ALIGNED_DAY_OF_WEEK_IN_YEAR`
- `DAY_OF_MONTH`
- `DAY_OF_YEAR`
- `EPOCH_DAY`
- `ALIGNED_WEEK_OF_MONTH`
- `ALIGNED_WEEK_OF_YEAR`
- `MONTH_OF_YEAR`
- `PROLEPTIC_MONTH`
- `YEAR_OF_ERA`
- `YEAR`
- `ERA`

All other `ChronoField` instances will return false.

If the field is not a `ChronoField`, then the result of this method is obtained by invoking `TemporalField.isSupportedBy(TemporalAccessor)` passing `this` as the argument. Whether the field is supported is determined by the field.

Specified by:

`isSupported` in interface `ChronoLocalDateTime<LocalDate>`

Specified by:

`isSupported` in interface `TemporalAccessor`

Parameters:

`field - the field to check, null returns false`

Returns:

true if the field is supported on this date-time, false if not

isSupported

public boolean isSupported(TemporalUnit unit)

Checks if the specified unit is supported.

This checks if the specified unit can be added to, or subtracted from, this date-time. If false, then calling the plus(long, TemporalUnit) and minus methods will throw an exception.

If the unit is a ChronoUnit then the query is implemented here. The supported units are:

- NANOS
- MICROS
- MILLIS
- SECONDS
- MINUTES
- HOURS
- HALF_DAYS
- DAYS
- WEEKS
- MONTHS
- YEARS
- DECADES
- CENTURIES
- MILLENNIA
- ERAS

All other ChronoUnit instances will return false.

If the unit is not a ChronoUnit, then the result of this method is obtained by invoking TemporalUnit.isSupportedBy(Temporal) passing this as the argument. Whether the unit is supported is determined by the unit.

Specified by:

isSupported in interface ChronoLocalDateTime<LocalDate>

Specified by:

isSupported in interface Temporal

Parameters:

unit - the unit to check, null returns false

Returns:

true if the unit can be added/subtracted, false if not

range

```
public ValueRange range(TemporalField field)
```

Gets the range of valid values for the specified field.

The range object expresses the minimum and maximum valid values for a field. This date-time is used to enhance the accuracy of the returned range. If it is not possible to return the range, because the field is not supported or for some other reason, an exception is thrown.

If the field is a `ChronoField` then the query is implemented here. The `supported fields` will return appropriate range instances. All other `ChronoField` instances will throw an `UnsupportedTemporalTypeException`.

If the field is not a `ChronoField`, then the result of this method is obtained by invoking `TemporalField.rangeRefinedBy(TemporalAccessor)` passing `this` as the argument. Whether the range can be obtained is determined by the field.

Specified by:

`range` in interface `TemporalAccessor`

Parameters:

`field - the field to query the range for, not null`

Returns:

`the range of valid values for the field, not null`

Throws:

`DateTimeException - if the range for the field cannot be obtained`

`UnsupportedTemporalTypeException - if the field is not supported`

get

```
public int get(TemporalField field)
```

Gets the value of the specified field from this date-time as an `int`.

This queries this date-time for the value of the specified field. The returned value will always be within the valid range of values for the field. If it is not possible to return the value, because the field is not supported or for some other reason, an exception is thrown.

If the field is a `ChronoField` then the query is implemented here. The `supported fields` will return valid values based on this date-time, except `NANO_OF_DAY`, `MICRO_OF_DAY`, `EPOCH_DAY` and `PROLEPTIC_MONTH` which are too large to fit in an `int` and throw a `DateTimeException`. All other `ChronoField` instances will throw an `UnsupportedTemporalTypeException`.

If the field is not a ChronoField, then the result of this method is obtained by invoking TemporalField.getFrom(TemporalAccessor) passing this as the argument. Whether the value can be obtained, and what the value represents, is determined by the field.

Specified by:

get in interface TemporalAccessor

Parameters:

field - the field to get, not null

Returns:

the value for the field

Throws:

DateTimeException - if a value for the field cannot be obtained or the value is outside the range of valid values for the field

UnsupportedTemporalTypeException - if the field is not supported or the range of values exceeds an int

ArithmeticException - if numeric overflow occurs

getLong

public long getLong(TemporalField field)

Gets the value of the specified field from this date-time as a long.

This queries this date-time for the value of the specified field. If it is not possible to return the value, because the field is not supported or for some other reason, an exception is thrown.

If the field is a ChronoField then the query is implemented here. The supported fields will return valid values based on this date-time. All other ChronoField instances will throw an UnsupportedTemporalTypeException.

If the field is not a ChronoField, then the result of this method is obtained by invoking TemporalField.getFrom(TemporalAccessor) passing this as the argument. Whether the value can be obtained, and what the value represents, is determined by the field.

Specified by:

getLong in interface TemporalAccessor

Parameters:

field - the field to get, not null

Returns:

the value for the field

Throws:

DateTimeException - if a value for the field cannot be obtained

UnsupportedTemporalTypeException - if the field is not supported

ArithmeticException - if numeric overflow occurs

toLocalDate

```
public LocalDate toLocalDate()
```

Gets the LocalDate part of this date-time.

This returns a LocalDate with the same year, month and day as this date-time.

Specified by:

toLocalDate in interface ChronoLocalDateTime<LocalDate>

Returns:

the date part of this date-time, not null

getYear

```
public int getYear()
```

Gets the year field.

This method returns the primitive int value for the year.

The year returned by this method is proleptic as per get(YEAR). To obtain the year-of-era, use get(YEAR_OF_ERA).

Returns:

the year, from MIN_YEAR to MAX_YEAR

getMonthValue

```
public int getMonthValue()
```

Gets the month-of-year field from 1 to 12.

This method returns the month as an `int` from 1 to 12. Application code is frequently clearer if the enum `Month` is used by calling `getMonth()`.

Returns:

`the month-of-year, from 1 to 12`

See Also:

`getMonth()`

getMonth

```
public Month getMonth()
```

Gets the month-of-year field using the `Month` enum.

This method returns the enum `Month` for the month. This avoids confusion as to what `int` values mean. If you need access to the primitive `int` value then the enum provides the `int value`.

Returns:

`the month-of-year, not null`

See Also:

`getMonthValue()`

getDayOfMonth

```
public int getDayOfMonth()
```

Gets the day-of-month field.

This method returns the primitive `int` value for the day-of-month.

Returns:

`the day-of-month, from 1 to 31`

getDayOfYear

```
public int getDayOfYear()
```

Gets the day-of-year field.

This method returns the primitive `int` value for the day-of-year.

Returns:

`the day-of-year, from 1 to 365, or 366 in a leap year`

getDayOfWeek

`public DayOfWeek getDayOfWeek()`

Gets the day-of-week field, which is an enum `DayOfWeek`.

This method returns the enum `DayOfWeek` for the day-of-week. This avoids confusion as to what `int` values mean. If you need access to the primitive `int` value then the enum provides the `int value`.

Additional information can be obtained from the `DayOfWeek`. This includes textual names of the values.

Returns:

`the day-of-week, not null`

toLocalTime

`public LocalTime toLocalTime()`

Gets the `LocalTime` part of this date-time.

This returns a `LocalTime` with the same hour, minute, second and nanosecond as this date-time.

Specified by:

`toLocalTime in interface ChronoLocalDateTime<LocalDate>`

Returns:

`the time part of this date-time, not null`

getHour

`public int getHour()`

Gets the hour-of-day field.

Returns:

`the hour-of-day, from 0 to 23`

getMinute

```
public int getMinute()
```

Gets the minute-of-hour field.

Returns:

`the minute-of-hour, from 0 to 59`

getSecond

```
public int getSecond()
```

Gets the second-of-minute field.

Returns:

`the second-of-minute, from 0 to 59`

getNano

```
public int getNano()
```

Gets the nano-of-second field.

Returns:

`the nano-of-second, from 0 to 999,999,999`

with

```
public LocalDateTime with(TemporalAdjuster adjuster)
```

Returns an adjusted copy of this date-time.

This returns a `LocalDateTime`, based on this one, with the date-time adjusted. The adjustment takes place using the specified adjuster strategy object. Read the documentation of the adjuster to understand what adjustment will be made.

A simple adjuster might simply set the one of the fields, such as the year field. A more complex adjuster might set the date to the last day of the month.

A selection of common adjustments is provided in `TemporalAdjusters`. These include finding the "last day of the month" and "next Wednesday". Key date-time classes also implement the `TemporalAdjuster` interface, such as `Month` and `MonthDay`. The adjuster is responsible for handling special cases, such as the varying lengths of month and leap years.

For example this code returns a date on the last day of July:

```
import static java.time.Month.*;
import static java.time.temporal.TemporalAdjusters.*;

result = localDateTime.with(JULY).with(lastDayOfMonth());
```

The classes `LocalDate` and `LocalTime` implement `TemporalAdjuster`, thus this method can be used to change the date, time or offset:

```
result = localDateTime.with(date);
result = localDateTime.with(time);
```

The result of this method is obtained by invoking the `TemporalAdjuster.adjustInto(Temporal)` method on the specified adjuster passing `this` as the argument.

This instance is immutable and unaffected by this method call.

Specified by:

`with` in interface `ChronoLocalDateTime<LocalDate>`

Specified by:

`with` in interface `Temporal`

Parameters:

`adjuster` - the adjuster to use, not null

Returns:

a LocalDateTime based on this with the adjustment made, not null

Throws:

`DateTimeException` - if the adjustment cannot be made

`ArithmeticException` - if numeric overflow occurs

with

```
public LocalDateTime with(TemporalField field,
                          long newValue)
```

Returns a copy of this date-time with the specified field set to a new value.

This returns a `LocalDateTime`, based on this one, with the value for the specified field changed. This can be used to change any supported field, such as the year, month or day-of-month. If it is not possible to set the value, because the field is not supported or for some other reason, an exception is thrown.

In some cases, changing the specified field can cause the resulting date-time to become invalid, such as changing the month from 31st January to February would make the day-of-month invalid. In cases like this, the field is responsible for resolving the date. Typically it will choose the previous valid date, which would be the last valid day of February in this example.

If the field is a `ChronoField` then the adjustment is implemented here. The `supported fields` will behave as per the matching method on `LocalDate` or `LocalTime`. All other `ChronoField` instances will throw an `UnsupportedTemporalTypeException`.

If the field is not a `ChronoField`, then the result of this method is obtained by invoking `TemporalField.adjustInto(Temporal, long)` passing `this` as the argument. In this case, the field determines whether and how to adjust the instant.

This instance is immutable and unaffected by this method call.

Specified by:

`with` in interface `ChronoLocalDateTime<LocalDate>`

Specified by:

`with` in interface `Temporal`

Parameters:

`field - the field to set in the result, not null`

`newValue - the new value of the field in the result`

Returns:

`a LocalDateTime based on this with the specified field set, not null`

Throws:

`DateTimeException - if the field cannot be set`

`UnsupportedTemporalTypeException - if the field is not supported`

`ArithmeticException - if numeric overflow occurs`

withYear

```
public LocalDateTime withYear(int year)
```

Returns a copy of this `LocalDateTime` with the year altered.

The time does not affect the calculation and will be the same in the result. If the day-of-month is invalid for the year, it will be changed to the last valid day of the month.

This instance is immutable and unaffected by this method call.

Parameters:

year - the year to set in the result, from MIN_YEAR to MAX_YEAR

Returns:

a LocalDateTime based on this date-time with the requested year, not null

Throws:

DateTimeException - if the year value is invalid

withMonth

public LocalDateTime withMonth(int month)

Returns a copy of this LocalDateTime with the month-of-year altered.

The time does not affect the calculation and will be the same in the result. If the day-of-month is invalid for the year, it will be changed to the last valid day of the month.

This instance is immutable and unaffected by this method call.

Parameters:

month - the month-of-year to set in the result, from 1 (January) to 12 (December)

Returns:

a LocalDateTime based on this date-time with the requested month, not null

Throws:

DateTimeException - if the month-of-year value is invalid

withDayOfMonth

public LocalDateTime withDayOfMonth(int dayOfMonth)

Returns a copy of this LocalDateTime with the day-of-month altered.

If the resulting date-time is invalid, an exception is thrown. The time does not affect the calculation and will be the same in the result.

This instance is immutable and unaffected by this method call.

Parameters:

dayOfMonth - the day-of-month to set in the result, from 1 to 28-31

Returns:

a LocalDateTime based on this date-time with the requested day, not null

Throws:

DateTimeException - if the day-of-month value is invalid, or if the day-of-month is invalid for the month-year

withDayOfYear

```
public LocalDateTime withDayOfYear(int dayOfYear)
```

Returns a copy of this LocalDateTime with the day-of-year altered.

If the resulting date-time is invalid, an exception is thrown.

This instance is immutable and unaffected by this method call.

Parameters:

dayOfYear - the day-of-year to set in the result, from 1 to 365-366

Returns:

a LocalDateTime based on this date with the requested day, not null

Throws:

DateTimeException - if the day-of-year value is invalid, or if the day-of-year is invalid for the year

withHour

```
public LocalDateTime withHour(int hour)
```

Returns a copy of this LocalDateTime with the hour-of-day altered.

This instance is immutable and unaffected by this method call.

Parameters:

hour - the hour-of-day to set in the result, from 0 to 23

Returns:

a LocalDateTime based on this date-time with the requested hour, not null

Throws:

DateTimeException - if the hour value is invalid

withMinute

```
public LocalDateTime withMinute(int minute)
```

Returns a copy of this LocalDateTime with the minute-of-hour altered.

This instance is immutable and unaffected by this method call.

Parameters:

minute - the minute-of-hour to set in the result, from 0 to 59

Returns:

a LocalDateTime based on this date-time with the requested minute, not null

Throws:

DateTimeException - if the minute value is invalid

withSecond

public LocalDateTime withSecond(int second)

Returns a copy of this LocalDateTime with the second-of-minute altered.

This instance is immutable and unaffected by this method call.

Parameters:

second - the second-of-minute to set in the result, from 0 to 59

Returns:

a LocalDateTime based on this date-time with the requested second, not null

Throws:

DateTimeException - if the second value is invalid

withNano

```
public LocalDateTime withNano(int nanoOfSecond)
```

Returns a copy of this `LocalDateTime` with the nano-of-second altered.

This instance is immutable and unaffected by this method call.

Parameters:

`nanoOfSecond - the nano-of-second to set in the result, from 0 to 999,999,999`

Returns:

`a LocalDateTime based on this date-time with the requested nanosecond, not null`

Throws:

`DateTimeException - if the nano value is invalid`

truncatedTo

```
public LocalDateTime truncatedTo(TemporalUnit unit)
```

Returns a copy of this `LocalDateTime` with the time truncated.

Truncation returns a copy of the original date-time with fields smaller than the specified unit set to zero. For example, truncating with the `minutes` unit will set the second-of-minute and nano-of-second field to zero.

The unit must have a duration that divides into the length of a standard day without remainder. This includes all supplied time units on `ChronoUnit` and `DAYS`. Other units throw an exception.

This instance is immutable and unaffected by this method call.

Parameters:

`unit - the unit to truncate to, not null`

Returns:

`a LocalDateTime based on this date-time with the time truncated, not null`

Throws:

`DateTimeException - if unable to truncate`

`UnsupportedTemporalTypeException - if the unit is not supported`

plus

```
public LocalDateTime plus(TemporalAmount amountToAdd)
```

Returns a copy of this date-time with the specified amount added.

This returns a `LocalDateTime`, based on this one, with the specified amount added. The amount is typically `Period` or `Duration` but may be any other type implementing the `TemporalAmount` interface.

The calculation is delegated to the amount object by calling `TemporalAmount.addTo(Temporal)`. The amount implementation is free to implement the addition in any way it wishes, however it typically calls back to `plus(long, TemporalUnit)`. Consult the documentation of the amount implementation to determine if it can be successfully added.

This instance is immutable and unaffected by this method call.

Specified by:

`plus` in interface `ChronoLocalDateTime<LocalDate>`

Specified by:

`plus` in interface `Temporal`

Parameters:

`amountToAdd` - the amount to add, not null

Returns:

a LocalDateTime based on this date-time with the addition made, not null

Throws:

`DateTimeException` - if the addition cannot be made

`ArithmeticException` - if numeric overflow occurs

plus

```
public LocalDateTime plus(long amountToAdd,
                          TemporalUnit unit)
```

Returns a copy of this date-time with the specified amount added.

This returns a `LocalDateTime`, based on this one, with the amount in terms of the unit added. If it is not possible to add the amount, because the unit is not supported or for some other reason, an exception is thrown.

If the field is a `ChronoUnit` then the addition is implemented here. Date units are added as per `LocalDate.plus(long, TemporalUnit)`. Time units are added as per `LocalTime.plus(long, TemporalUnit)` with any overflow in days added equivalent to using `plusDays(long)`.

If the field is not a `ChronoUnit`, then the result of this method is obtained by invoking `TemporalUnit.addTo(Temporal, long)` passing `this` as the argument. In this case, the unit determines whether and how to perform the addition.

This instance is immutable and unaffected by this method call.

Specified by:

`plus` in interface `ChronoLocalDateTime<LocalDate>`

Specified by:

`plus` in interface `Temporal`

Parameters:

`amountToAdd` - the amount of the unit to add to the result, may be negative

`unit` - the unit of the amount to add, not null

Returns:

a LocalDateTime based on this date-time with the specified amount added, not null

Throws:

`DateTimeException` - if the addition cannot be made

`UnsupportedTemporalTypeException` - if the unit is not supported

`ArithmeticException` - if numeric overflow occurs

plusYears

```
public LocalDateTime plusYears(long years)
```

Returns a copy of this `LocalDateTime` with the specified number of years added.

This method adds the specified amount to the years field in three steps:

1. Add the input years to the year field
2. Check if the resulting date would be invalid
3. Adjust the day-of-month to the last valid day if necessary

For example, 2008-02-29 (leap year) plus one year would result in the invalid date 2009-02-29 (standard year). Instead of returning an invalid result, the last valid day of the month, 2009-02-28, is selected instead.

This instance is immutable and unaffected by this method call.

Parameters:

`years` - the years to add, may be negative

Returns:

a LocalDateTime based on this date-time with the years added, not null

Throws:

DateTimeException - if the result exceeds the supported date range

plusMonths

```
public LocalDateTime plusMonths(long months)
```

Returns a copy of this LocalDateTime with the specified number of months added.

This method adds the specified amount to the months field in three steps:

1. Add the input months to the month-of-year field
2. Check if the resulting date would be invalid
3. Adjust the day-of-month to the last valid day if necessary

For example, 2007-03-31 plus one month would result in the invalid date 2007-04-31. Instead of returning an invalid result, the last valid day of the month, 2007-04-30, is selected instead.

This instance is immutable and unaffected by this method call.

Parameters:

months - the months to add, may be negative

Returns:

a LocalDateTime based on this date-time with the months added, not null

Throws:

DateTimeException - if the result exceeds the supported date range

plusWeeks

```
public LocalDateTime plusWeeks(long weeks)
```

Returns a copy of this LocalDateTime with the specified number of weeks added.

This method adds the specified amount in weeks to the days field incrementing the month and year fields as necessary to ensure the result remains valid. The result is only invalid if the maximum/minimum year is exceeded.

For example, 2008-12-31 plus one week would result in 2009-01-07.

This instance is immutable and unaffected by this method call.

Parameters:

`weeks - the weeks to add, may be negative`

Returns:

`a LocalDateTime based on this date-time with the weeks added, not null`

Throws:

`DateTimeException - if the result exceeds the supported date range`

plusDays

`public LocalDateTime plusDays(long days)`

Returns a copy of this `LocalDateTime` with the specified number of days added.

This method adds the specified amount to the days field incrementing the month and year fields as necessary to ensure the result remains valid. The result is only invalid if the maximum/minimum year is exceeded.

For example, 2008-12-31 plus one day would result in 2009-01-01.

This instance is immutable and unaffected by this method call.

Parameters:

`days - the days to add, may be negative`

Returns:

`a LocalDateTime based on this date-time with the days added, not null`

Throws:

`DateTimeException - if the result exceeds the supported date range`

plusHours

`public LocalDateTime plusHours(long hours)`

Returns a copy of this `LocalDateTime` with the specified number of hours added.

This instance is immutable and unaffected by this method call.

Parameters:

`hours - the hours to add, may be negative`

Returns:

`a LocalDateTime based on this date-time with the hours added, not null`

Throws:

`DateTimeException` - if the result exceeds the supported date range

plusMinutes

```
public LocalDateTime plusMinutes(long minutes)
```

Returns a copy of this `LocalDateTime` with the specified number of minutes added.

This instance is immutable and unaffected by this method call.

Parameters:

`minutes` - the minutes to add, may be negative

Returns:

a `LocalDateTime` based on this date-time with the minutes added, not null

Throws:

`DateTimeException` - if the result exceeds the supported date range

plusSeconds

```
public LocalDateTime plusSeconds(long seconds)
```

Returns a copy of this `LocalDateTime` with the specified number of seconds added.

This instance is immutable and unaffected by this method call.

Parameters:

`seconds` - the seconds to add, may be negative

Returns:

a `LocalDateTime` based on this date-time with the seconds added, not null

Throws:

`DateTimeException` - if the result exceeds the supported date range

plusNanos

```
public LocalDateTime plusNanos(long nanos)
```

Returns a copy of this `LocalDateTime` with the specified number of nanoseconds added.

This instance is immutable and unaffected by this method call.

Parameters:

`nanos - the nanos to add, may be negative`

Returns:

`a LocalDateTime based on this date-time with the nanoseconds added, not null`

Throws:

`DateTimeException - if the result exceeds the supported date range`

minus

```
public LocalDateTime minus(TemporalAmount amountToSubtract)
```

Returns a copy of this date-time with the specified amount subtracted.

This returns a `LocalDateTime`, based on this one, with the specified amount subtracted. The amount is typically `Period` or `Duration` but may be any other type implementing the `TemporalAmount` interface.

The calculation is delegated to the amount object by calling `TemporalAmount.subtractFrom(Temporal)`. The amount implementation is free to implement the subtraction in any way it wishes, however it typically calls back to `minus(long, TemporalUnit)`. Consult the documentation of the amount implementation to determine if it can be successfully subtracted.

This instance is immutable and unaffected by this method call.

Specified by:

`minus in interface ChronoLocalDateTime<LocalDate>`

Specified by:

`minus in interface Temporal`

Parameters:

`amountToSubtract - the amount to subtract, not null`

Returns:

`a LocalDateTime based on this date-time with the subtraction made, not null`

Throws:

`DateTimeException - if the subtraction cannot be made`

`ArithmeticException - if numeric overflow occurs`

minus

```
public LocalDateTime minus(long amountToSubtract,
                           TemporalUnit unit)
```

Returns a copy of this date-time with the specified amount subtracted.

This returns a `LocalDateTime`, based on this one, with the amount in terms of the unit subtracted. If it is not possible to subtract the amount, because the unit is not supported or for some other reason, an exception is thrown.

This method is equivalent to `plus(long, TemporalUnit)` with the amount negated. See that method for a full description of how addition, and thus subtraction, works.

This instance is immutable and unaffected by this method call.

Specified by:

`minus` in interface `ChronoLocalDateTime<LocalDate>`

Specified by:

`minus` in interface `Temporal`

Parameters:

`amountToSubtract` - the amount of the unit to subtract from the result, may be negative

`unit` - the unit of the amount to subtract, not null

Returns:

a LocalDateTime based on this date-time with the specified amount subtracted, not null

Throws:

`DateTimeException` - if the subtraction cannot be made

`UnsupportedTemporalTypeException` - if the unit is not supported

`ArithmeticException` - if numeric overflow occurs

minusYears

```
public LocalDateTime minusYears(long years)
```

Returns a copy of this `LocalDateTime` with the specified number of years subtracted.

This method subtracts the specified amount from the years field in three steps:

1. Subtract the input years from the year field
2. Check if the resulting date would be invalid

3. Adjust the day-of-month to the last valid day if necessary

For example, 2008-02-29 (leap year) minus one year would result in the invalid date 2009-02-29 (standard year). Instead of returning an invalid result, the last valid day of the month, 2009-02-28, is selected instead.

This instance is immutable and unaffected by this method call.

Parameters:

`years - the years to subtract, may be negative`

Returns:

`a LocalDateTime based on this date-time with the years subtracted, not null`

Throws:

`DateTimeException - if the result exceeds the supported date range`

minusMonths

```
public LocalDateTime minusMonths(long months)
```

Returns a copy of this `LocalDateTime` with the specified number of months subtracted.

This method subtracts the specified amount from the months field in three steps:

1. Subtract the input months from the month-of-year field
2. Check if the resulting date would be invalid
3. Adjust the day-of-month to the last valid day if necessary

For example, 2007-03-31 minus one month would result in the invalid date 2007-04-31. Instead of returning an invalid result, the last valid day of the month, 2007-04-30, is selected instead.

This instance is immutable and unaffected by this method call.

Parameters:

`months - the months to subtract, may be negative`

Returns:

`a LocalDateTime based on this date-time with the months subtracted, not null`

Throws:

`DateTimeException - if the result exceeds the supported date range`

minusWeeks

```
public LocalDateTime minusWeeks(long weeks)
```

Returns a copy of this `LocalDateTime` with the specified number of weeks subtracted.

This method subtracts the specified amount in weeks from the days field decrementing the month and year fields as necessary to ensure the result remains valid. The result is only invalid if the maximum/minimum year is exceeded.

For example, 2009-01-07 minus one week would result in 2008-12-31.

This instance is immutable and unaffected by this method call.

Parameters:

`weeks - the weeks to subtract, may be negative`

Returns:

`a LocalDateTime based on this date-time with the weeks subtracted, not null`

Throws:

`DateTimeException - if the result exceeds the supported date range`

minusDays

```
public LocalDateTime minusDays(long days)
```

Returns a copy of this `LocalDateTime` with the specified number of days subtracted.

This method subtracts the specified amount from the days field decrementing the month and year fields as necessary to ensure the result remains valid. The result is only invalid if the maximum/minimum year is exceeded.

For example, 2009-01-01 minus one day would result in 2008-12-31.

This instance is immutable and unaffected by this method call.

Parameters:

`days - the days to subtract, may be negative`

Returns:

`a LocalDateTime based on this date-time with the days subtracted, not null`

Throws:

`DateTimeException - if the result exceeds the supported date range`

minusHours

```
public LocalDateTime minusHours(long hours)
```

Returns a copy of this `LocalDateTime` with the specified number of hours subtracted.

This instance is immutable and unaffected by this method call.

Parameters:

`hours - the hours to subtract, may be negative`

Returns:

`a LocalDateTime based on this date-time with the hours subtracted, not null`

Throws:

`DateTimeException - if the result exceeds the supported date range`

minusMinutes

```
public LocalDateTime minusMinutes(long minutes)
```

Returns a copy of this `LocalDateTime` with the specified number of minutes subtracted.

This instance is immutable and unaffected by this method call.

Parameters:

`minutes - the minutes to subtract, may be negative`

Returns:

`a LocalDateTime based on this date-time with the minutes subtracted, not null`

Throws:

`DateTimeException - if the result exceeds the supported date range`

minusSeconds

```
public LocalDateTime minusSeconds(long seconds)
```

Returns a copy of this `LocalDateTime` with the specified number of seconds subtracted.

This instance is immutable and unaffected by this method call.

Parameters:

`seconds - the seconds to subtract, may be negative`

Returns:

a LocalDateTime based on this date-time with the seconds subtracted, not null

Throws:

DateTimeException - if the result exceeds the supported date range

minusNanos

```
public LocalDateTime minusNanos(long nanos)
```

Returns a copy of this LocalDateTime with the specified number of nanoseconds subtracted.

This instance is immutable and unaffected by this method call.

Parameters:

nanos - the nanos to subtract, may be negative

Returns:

a LocalDateTime based on this date-time with the nanoseconds subtracted, not null

Throws:

DateTimeException - if the result exceeds the supported date range

query

```
public <R> R query(TemporalQuery<R> query)
```

Queries this date-time using the specified query.

This queries this date-time using the specified query strategy object. The TemporalQuery object defines the logic to be used to obtain the result. Read the documentation of the query to understand what the result of this method will be.

The result of this method is obtained by invoking the TemporalQuery.queryFrom(TemporalAccessor) method on the specified query passing this as the argument.

Specified by:

query in interface ChronoLocalDateTime<LocalDate>

Specified by:

query in interface TemporalAccessor

Type Parameters:

R - the type of the result

Parameters:

query - the query to invoke, not null

Returns:

the query result, null may be returned (defined by the query)

Throws:

DateTimeException - if unable to query (defined by the query)

ArithmeticException - if numeric overflow occurs (defined by the query)

adjustInto

```
public Temporal adjustInto(Temporal temporal)
```

Adjusts the specified temporal object to have the same date and time as this object.

This returns a temporal object of the same observable type as the input with the date and time changed to be the same as this.

The adjustment is equivalent to using `Temporal.with(TemporalField, long)` twice, passing `ChronoField.EPOCH_DAY` and `ChronoField.NANO_OF_DAY` as the fields.

In most cases, it is clearer to reverse the calling pattern by using `Temporal.with(TemporalAdjuster)`:

```
// these two lines are equivalent, but the second approach is recommended
temporal = thisLocalDateTime.adjustInto(temporal);
temporal = temporal.with(thisLocalDateTime);
```

This instance is immutable and unaffected by this method call.

Specified by:

adjustInto in interface ChronoLocalDateTime<LocalDate>

Specified by:

adjustInto in interface TemporalAdjuster

Parameters:

temporal - the target object to be adjusted, not null

Returns:

the adjusted object, not null

Throws:

DateTimeException - if unable to make the adjustment

ArithmeticException - if numeric overflow occurs

until

```
public long until(Temporal endExclusive,
                  TemporalUnit unit)
```

Calculates the amount of time until another date-time in terms of the specified unit.

This calculates the amount of time between two `LocalDateTime` objects in terms of a single `TemporalUnit`. The start and end points are `this` and the specified date-time. The result will be negative if the end is before the start. The `Temporal` passed to this method is converted to a `LocalDateTime` using `from(TemporalAccessor)`. For example, the amount in days between two date-times can be calculated using `startDateTime.until(endDateTime, DAYS)`.

The calculation returns a whole number, representing the number of complete units between the two date-times. For example, the amount in months between 2012-06-15T00:00 and 2012-08-14T23:59 will only be one month as it is one minute short of two months.

There are two equivalent ways of using this method. The first is to invoke this method. The second is to use `TemporalUnit.between(Temporal, Temporal)`:

```
// these two lines are equivalent
amount = start.until(end, MONTHS);
amount = MONTHS.between(start, end);
```

The choice should be made based on which makes the code more readable.

The calculation is implemented in this method for `ChronoUnit`. The units `NANOS`, `MICROS`, `MILLIS`, `SECONDS`, `MINUTES`, `HOURS` and `HALF_DAYS`, `DAYS`, `WEEKS`, `MONTHS`, `YEARS`, `DECADES`, `CENTURIES`, `MILLENNIA` and `ERAS` are supported. Other `ChronoUnit` values will throw an exception.

If the unit is not a `ChronoUnit`, then the result of this method is obtained by invoking `TemporalUnit.between(Temporal, Temporal)` passing `this` as the first argument and the converted input temporal as the second argument.

This instance is immutable and unaffected by this method call.

Specified by:

`until` in interface `Temporal`

Parameters:

`endExclusive - the end date, exclusive, which is converted to a LocalDateTime, not null`

`unit - the unit to measure the amount in, not null`

Returns:

the amount of time between this date-time and the end date-time

Throws:

DateTimeException - if the amount cannot be calculated, or the end temporal cannot be converted to a LocalDateTime

UnsupportedTemporalTypeException - if the unit is not supported

ArithmeticException - if numeric overflow occurs

format

public String format(DateTimeFormatter formatter)

Formats this date-time using the specified formatter.

This date-time will be passed to the formatter to produce a string.

Specified by:

format in interface ChronoLocalDateTime<LocalDate>

Parameters:

formatter - the formatter to use, not null

Returns:

the formatted date-time string, not null

Throws:

DateTimeException - if an error occurs during printing

atOffset

public OffsetDateTime atOffset(ZoneOffset offset)

Combines this date-time with an offset to create an OffsetDateTime.

This returns an OffsetDateTime formed from this date-time at the specified offset. All possible combinations of date-time and offset are valid.

Parameters:

offset - the offset to combine with, not null

Returns:

the offset date-time formed from this date-time and the specified offset, not null

atZone

`public ZonedDateTime atZone(ZoneId zone)`

Combines this date-time with a time-zone to create a `ZonedDateTime`.

This returns a `ZonedDateTime` formed from this date-time at the specified time-zone. The result will match this date-time as closely as possible. Time-zone rules, such as daylight savings, mean that not every local date-time is valid for the specified zone, thus the local date-time may be adjusted.

The local date-time is resolved to a single instant on the time-line. This is achieved by finding a valid offset from UTC/Greenwich for the local date-time as defined by the `rules` of the zone ID.

In most cases, there is only one valid offset for a local date-time. In the case of an overlap, where clocks are set back, there are two valid offsets. This method uses the earlier offset typically corresponding to "summer".

In the case of a gap, where clocks jump forward, there is no valid offset. Instead, the local date-time is adjusted to be later by the length of the gap. For a typical one hour daylight savings change, the local date-time will be moved one hour later into the offset typically corresponding to "summer".

To obtain the later offset during an overlap, call `ZonedDateTime.withLaterOffsetAtOverlap()` on the result of this method. To throw an exception when there is a gap or overlap, use `ZonedDateTime.ofStrict(LocalDateTime, ZoneOffset, ZoneId)`.

Specified by:

`atZone in interface ChronoLocalDateTime<LocalDate>`

Parameters:

`zone - the time-zone to use, not null`

Returns:

`the zoned date-time formed from this date-time, not null`

compareTo

`public int compareTo(ChronoLocalDateTime<?> other)`

Compares this date-time to another date-time.

The comparison is primarily based on the date-time, from earliest to latest. It is "consistent with equals", as defined by `Comparable`.

If all the date-times being compared are instances of `LocalDateTime`, then the comparison will be entirely based on the date-time. If some dates being compared are in different chronologies, then the chronology is also

considered, see ChronoLocalDateTime.compareTo(java.time.chrono.ChronoLocalDateTime<?>).

Specified by:

compareTo in interface Comparable<ChronoLocalDateTime<?>>

Specified by:

compareTo in interface ChronoLocalDateTime<LocalDate>

Parameters:

other - the other date-time to compare to, not null

Returns:

the comparator value, negative if less, positive if greater

isAfter

public boolean isAfter(ChronoLocalDateTime<?> other)

Checks if this date-time is after the specified date-time.

This checks to see if this date-time represents a point on the local time-line after the other date-time.

```
   LocalDate a = LocalDateTime.of(2012, 6, 30, 12, 00);
   LocalDate b = LocalDateTime.of(2012, 7, 1, 12, 00);
   a.isAfter(b) == false
   a.isAfter(a) == false
   b.isAfter(a) == true
```

This method only considers the position of the two date-times on the local time-line. It does not take into account the chronology, or calendar system. This is different from the comparison in compareTo(ChronoLocalDateTime), but is the same approach as ChronoLocalDateTime.timeLineOrder().

Specified by:

isAfter in interface ChronoLocalDateTime<LocalDate>

Parameters:

other - the other date-time to compare to, not null

Returns:

true if this date-time is after the specified date-time

isBefore

public boolean isBefore(ChronoLocalDateTime<?> other)

Checks if this date-time is before the specified date-time.

This checks to see if this date-time represents a point on the local time-line before the other date-time.

```
LocalDate a = LocalDateTime.of(2012, 6, 30, 12, 00);
LocalDate b = LocalDateTime.of(2012, 7, 1, 12, 00);
a.isBefore(b) == true
a.isBefore(a) == false
b.isBefore(a) == false
```

This method only considers the position of the two date-times on the local time-line. It does not take into account the chronology, or calendar system. This is different from the comparison in `compareTo(ChronoLocalDateTime)`, but is the same approach as `ChronoLocalDateTime.timeLineOrder()`.

Specified by:

`isBefore` in interface `ChronoLocalDateTime<LocalDate>`

Parameters:

`other - the other date-time to compare to, not null`

Returns:

`true if this date-time is before the specified date-time`

isEqual

```
public boolean isEqual(ChronoLocalDateTime<?> other)
```

Checks if this date-time is equal to the specified date-time.

This checks to see if this date-time represents the same point on the local time-line as the other date-time.

```
LocalDate a = LocalDateTime.of(2012, 6, 30, 12, 00);
LocalDate b = LocalDateTime.of(2012, 7, 1, 12, 00);
a.isEqual(b) == false
a.isEqual(a) == true
b.isEqual(a) == false
```

This method only considers the position of the two date-times on the local time-line. It does not take into account the chronology, or calendar system. This is different from the comparison in `compareTo(ChronoLocalDateTime)`, but is the same approach as `ChronoLocalDateTime.timeLineOrder()`.

Specified by:

`isEqual` in interface `ChronoLocalDateTime<LocalDate>`

Parameters:

other - the other date-time to compare to, not null

Returns:

true if this date-time is equal to the specified date-time

equals

public boolean equals(Object obj)

Checks if this date-time is equal to another date-time.

Compares this LocalDateTime with another ensuring that the date-time is the same. Only objects of type LocalDateTime are compared, other types return false.

Specified by:

equals in interface ChronoLocalDateTime<LocalDate>

Overrides:

equals in class Object

Parameters:

obj - the object to check, null returns false

Returns:

true if this is equal to the other date-time

See Also:

Object.hashCode(), HashMap

hashCode

public int hashCode()

A hash code for this date-time.

Specified by:

hashCode in interface ChronoLocalDateTime<LocalDate>

Overrides:

hashCode in class Object

Returns:

a suitable hash code

See Also:

Object.equals(java.lang.Object), System.identityHashCode(java.lang.Object)

toString

```
public String toString()
```

Outputs this date-time as a `String`, such as `2007-12-03T10:15:30`.

The output will be one of the following ISO-8601 formats:

- `uuuu-MM-dd'T'HH:mm`
- `uuuu-MM-dd'T'HH:mm:ss`
- `uuuu-MM-dd'T'HH:mm:ss.SSS`
- `uuuu-MM-dd'T'HH:mm:ss.SSSSSS`
- `uuuu-MM-dd'T'HH:mm:ss.SSSSSSSSS`

The format used will be the shortest that outputs the full value of the time where the omitted parts are implied to be zero.

Specified by:

`toString` in interface `ChronoLocalDateTime<LocalDate>`

Overrides:

`toString` in class `Object`

Returns:

a string representation of this date-time, not null

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.